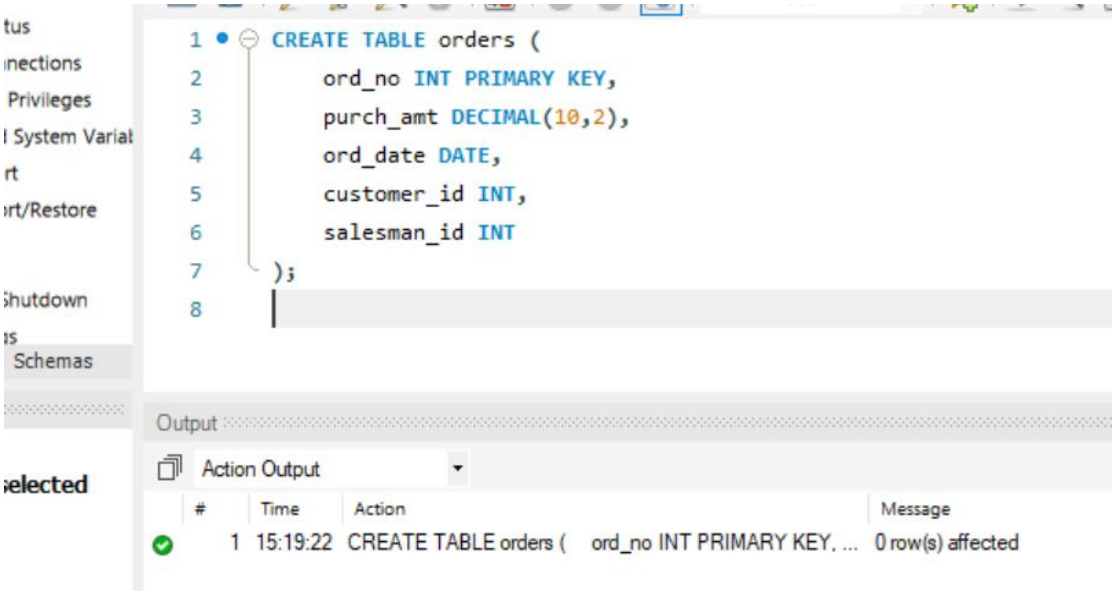
Database System Engineering and Distributed Backend Development
Course Code: 25CS1302E

2520030198
K.Vaishnavi

STEP 1: CREATE



STEP 2; INSERT

```
8  INSERT INTO orders VALUES
9  (70001,150.50,'2012-10-05',3005,5002),
10 (70009,270.65,'2012-09-10',3001,5005),
11 (70002,65.26,'2012-10-05',3002,5001),
12 (70004,110.50,'2012-08-17',3009,5003),
13 (70007,948.50,'2012-09-10',3005,5002),
14 (70005,2400.60,'2012-07-27',3007,5001),
15 (70008,5760.00,'2012-09-10',3002,5001),
16 (70010,1983.43,'2012-10-10',3004,5006),
17 (70003,2480.40,'2012-10-10',3009,5003),
18 (70012,250.45,'2012-06-27',3008,5002),
19 (70011,75.29,'2012-08-17',3003,5007),
20 (70013,3045.60,'2012-04-25',3002,5001);
```

Output

Action Output

#	Time	Action	Message
1	15:19:22	CREATE TABLE orders (ord_no INT PRIMARY KEY, ...	0 row(s) aff
2	15:20:36	INSERT INTO orders VALUES (70001,150.50,'2012-10-...	12 row(s) a

STEP 3: SELECT

```
21  SELECT * FROM orders;
22
23
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1983.43	2012-10-10	3004	5006
70011	75.29	2012-08-17	3003	5007
70012	250.45	2012-06-27	3008	5002
70013	3045.60	2012-04-25	3002	5001
NULL	NULL	NULL	NULL	NULL

orders 60

Step4: WHERE CLAUSE

```
22
23 •  SELECT *
24     FROM orders
25     WHERE purch_amt > 2000;
26
```

Result Grid | Filter Rows: | Edit:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
NULL	NULL	NULL	NULL	NULL

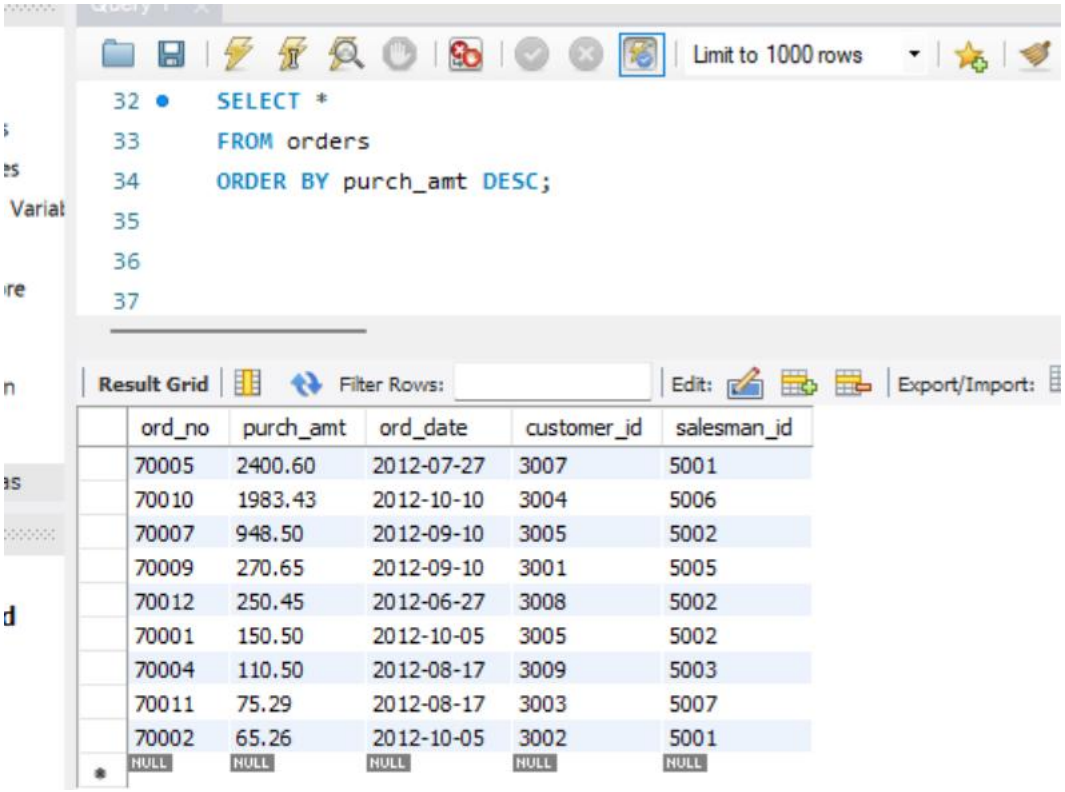
STEP 5: WHERE CLAUSE

```
27
28 •  SELECT *
29     FROM orders
30     WHERE ord_date='2012-09-10';
31
32
```

Result Grid | Filter Rows: | Edit: | Expor

ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
NULL	NULL	NULL	NULL	NULL

STEP : ORDER BY



```
SELECT *
FROM orders
ORDER BY purch_amt DESC;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70002	65.26	2012-10-05	3002	5001
NULL	NULL	NULL	NULL	NULL

STEP 7 : ORDER BY

```
SELECT *
FROM orders
ORDER BY ord_date;
```

Result Grid | Filter Rows: | Edit:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.60	2012-04-25	3002	5001
70012	250.45	2012-06-27	3008	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003

STEP 8: SUM ()

```
40
41 •  SELECT SUM(purch_amt) AS total_revenue
42    FROM orders;
43
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_revenue
17541.18

STEP 9: AVG ()

```
43 •  SELECT AVG(purch_amt) AS average_order
44    FROM orders;
45
```

Result Grid | Filter Rows: | Export: | Wrap C

average_order
1461.765000

STEP 10: MAX()

```
45 •  SELECT MAX(purch_amt) AS highest_order
46    FROM orders;
47
```

Result Grid | Filter Rows: | Export: | W

highest_order
5760.00

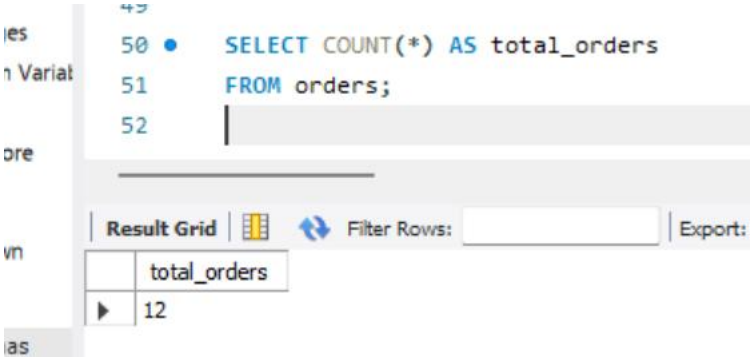
STEP 11: MIN ()

```
47 •  SELECT MIN(purch_amt) AS lowest_order
48    FROM orders;
49
```

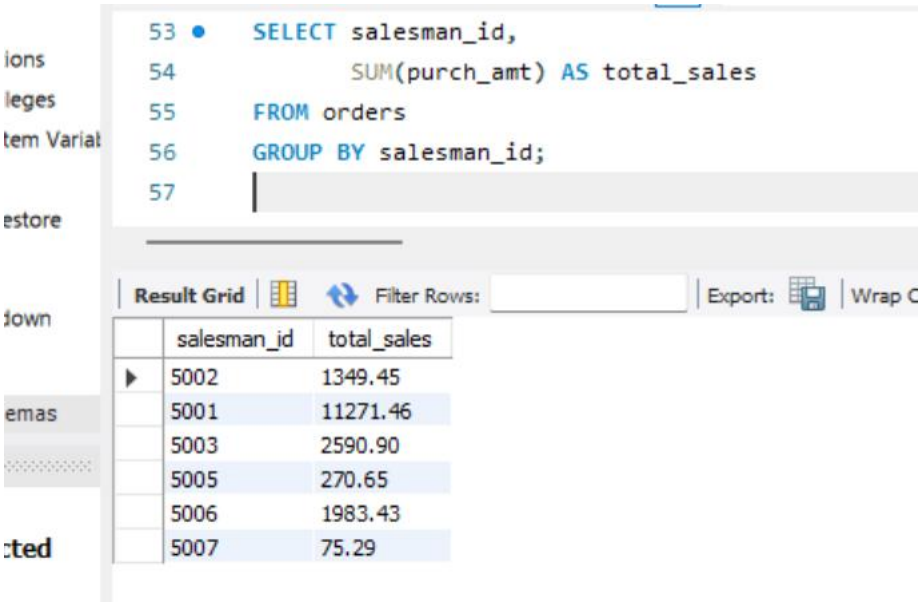
Result Grid | Filter Rows: | Export: | W

lowest_order
65.26

STEP 12: COUNT ()



STEP 13: GROUP BY



STEP 14: GROUP BY

```
57 •  SELECT customer_id,
58            SUM(purch_amt) AS total_purchase
59    FROM orders
60    GROUP BY customer_id;
61
62
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

STEP 15: GROUP BY + MAX ()

```
62 •  SELECT customer_id,
63            MAX(purch_amt) AS highest_purchase
64    FROM orders
65    GROUP BY customer_id;
66
67
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

customer_id	highest_purchase
3005	948.50
3002	5760.00
3009	2480.40
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

STEP 1: GROUP BY + HAVING

```
66 •  SELECT salesman_id,
67            SUM(purch_amt) AS total_sales
68    FROM orders
69    GROUP BY salesman_id
70    HAVING SUM(purch_amt) > 3000;
```

Result Grid | Filter Rows: | Export:

salesman_id	total_sales
5001	11271.46

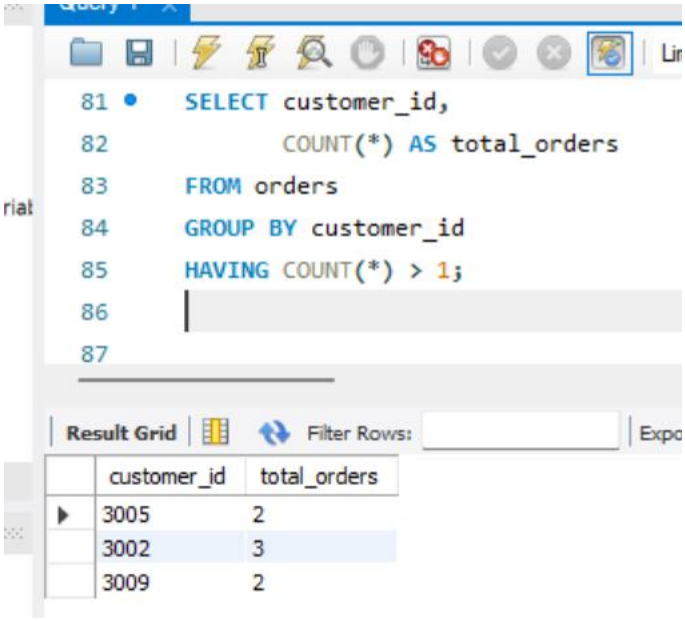
STEP 17: GROUP BY + HAVING

```
73 •  SELECT customer_id,
74            SUM(purch_amt) AS total_purchase
75    FROM orders
76    GROUP BY customer_id
77    HAVING SUM(purch_amt) > 2500;
78
79
```

Result Grid | Filter Rows: | Export:

customer_id	total_purchase
3002	8870.86
3009	2590.90

STEP 18: GROUP BY + HAVING

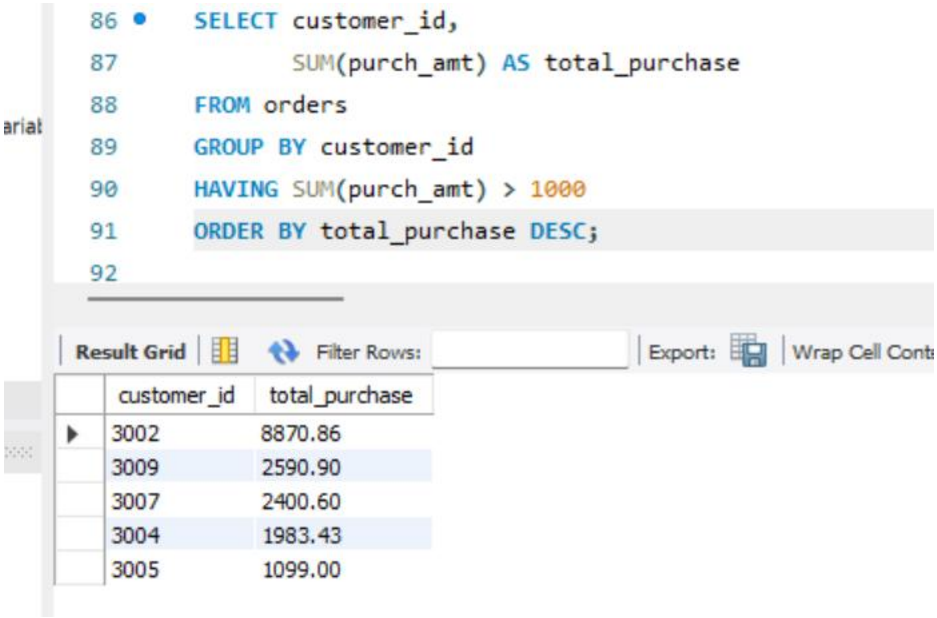


```
81 •  SELECT customer_id,
82            COUNT(*) AS total_orders
83    FROM orders
84    GROUP BY customer_id
85    HAVING COUNT(*) > 1;
86
87
```

Result Grid | Filter Rows: | Expo

customer_id	total_orders
3005	2
3002	3
3009	2

STEP 19: GROUP BY + HAVING + ORDER BY



```
86 •  SELECT customer_id,
87            SUM(purch_amt) AS total_purchase
88    FROM orders
89    GROUP BY customer_id
90    HAVING SUM(purch_amt) > 1000
91    ORDER BY total_purchase DESC;
92
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

customer_id	total_purchase
3002	8870.86
3009	2590.90
3007	2400.60
3004	1983.43
3005	1099.00

STEP 20: BETWEEN + HAVING

```
94 •  SELECT customer_id,
95            MAX(purch_amt) AS max_purchase
96    FROM orders
97    GROUP BY customer_id
98    HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
99
100
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

STEP 21: COUNT + HAVING

```
99 •  SELECT salesman_id,
100           COUNT(*) AS total_orders
101   FROM orders
102   GROUP BY salesman_id
103   HAVING COUNT(*) >= 2;
104
```

Result Grid | Filter Rows: | Expor

salesman_id	total_orders
5002	3
5001	4
5003	2

STEP 22: MAX + GROUP BY + HAVING

```
104 •   SELECT ord_date,
105             MAX(purch_amt) AS highest_purchase
106     FROM orders
107     GROUP BY ord_date
108     HAVING MAX(purch_amt) > 2000;
109
```

Result Grid | Filter Rows: | Export: | Wrap

ord_date	highest_purchase
2012-10-10	2480.40
2012-07-27	2400.60
2012-09-10	5760.00
2012-04-25	3045.60